

Problem 2.10.52

ee25btech11023-Venkata Sai

September 1, 2025

1 Problem

2 Solution

- Formula
- Method of finding the projection
- Finding projection
- Conclusion
- Plot

3 C Code

4 Python Code

Problem Statement

Let $\mathbf{a} = \hat{\mathbf{i}} + 2\hat{\mathbf{j}} + \hat{\mathbf{k}}$, $\mathbf{b} = \hat{\mathbf{i}} - \hat{\mathbf{j}} + \hat{\mathbf{k}}$ and $\mathbf{c} = \hat{\mathbf{i}} + \hat{\mathbf{j}} - \hat{\mathbf{k}}$. A vector in the plane of $\mathbf{a}$ and $\mathbf{b}$ whose projection on $\mathbf{c}$ is $\frac{1}{\sqrt{3}}$, is

Formula

Let $\mathbf{v}$ be the vector in the plane of $\mathbf{a}$ and $\mathbf{b}$

$$[\mathbf{v} \quad \mathbf{a} \quad \mathbf{b}] = 0 \implies \mathbf{v} = \alpha \mathbf{a} + \beta \mathbf{b} \quad (1.1)$$

where

$$\mathbf{a} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}, \mathbf{b} = \begin{pmatrix} 1 \\ -1 \\ 1 \end{pmatrix} \quad (1.2)$$

Then

$$\mathbf{v} = \alpha \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} + \beta \begin{pmatrix} 1 \\ -1 \\ 1 \end{pmatrix} = \begin{pmatrix} \alpha + \beta \\ 2\alpha - \beta \\ \alpha + \beta \end{pmatrix}. \quad (1.3)$$

Method of finding the projection

Mathematically, the projection of $\mathbf{v}$ on $\mathbf{c}$ is defined as

$$\mathbf{d} = k\mathbf{c}, \text{ such that } (\mathbf{v} - \mathbf{d})^\top \mathbf{d} = 0 \quad (1.4)$$

yielding

$$(\mathbf{v} - k\mathbf{c})^\top \mathbf{d} = 0 \quad (1.5)$$

$$\text{or, } k = \frac{\mathbf{v}^\top \mathbf{c}}{\|\mathbf{c}\|^2}, \implies \mathbf{d} = \frac{\mathbf{v}^\top \mathbf{c}}{\|\mathbf{c}\|^2} \mathbf{c}. \quad (1.6)$$

Finding projection

$$\|\mathbf{c}\|^2 = \mathbf{c}^\top \mathbf{c} = \begin{pmatrix} 1 & 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ -1 \end{pmatrix} \quad (1.7)$$

$$= 1 + 1 + 1 = 3 \implies \|\mathbf{c}\| = \sqrt{3} \quad (1.8)$$

$$\mathbf{v}^\top \mathbf{c} = \begin{pmatrix} \alpha + \beta & 2\alpha - \beta & \alpha + \beta \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ -1 \end{pmatrix} \quad (1.9)$$

$$= \alpha + \beta + 2\alpha - \beta - \alpha - \beta = 2\alpha - \beta \quad (1.10)$$

$$\mathbf{d} = \frac{2\alpha - \beta}{3} \mathbf{c} \quad (1.11)$$

Conclusion

Given $\|d\| = \frac{1}{\sqrt{3}}$, Then

$$\frac{|2\alpha - \beta|}{3} \|\mathbf{c}\| = \frac{|2\alpha - \beta|}{3} \sqrt{3} = \frac{1}{\sqrt{3}} \implies |2\alpha - \beta| = 1 \quad (1.12)$$

From equation(3) we know that that x and z coordinates in our required vector must be equal.

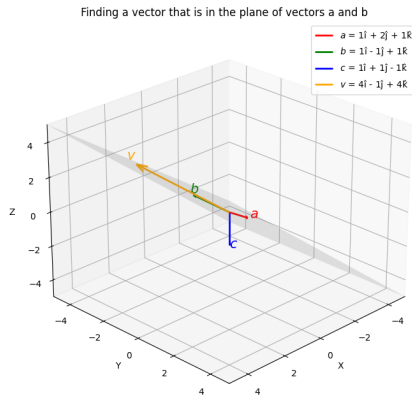
If $2\alpha - \beta = -1$, from the most reliable option i.e option(1) which has the same x and z coordinates, we get

$$\alpha + \beta = 4 \text{ and } 2\alpha - \beta = -1 \quad (1.13)$$

$$3\alpha = 3 \implies \alpha = 1 \text{ and } \beta = 3 \implies \mathbf{v} = \begin{pmatrix} 4 \\ -1 \\ 4 \end{pmatrix} \quad (1.14)$$

If $2\alpha - \beta = 1$, from the options x and z coordinates are not equal, No option satisfy.

Plot



Figure

C Code

```
void get_vectors(double* vector_data) {  
    vector_data[0] = 1.0; vector_data[1] = 2.0; vector_data[2] =  
        1.0;  
    vector_data[3] = 1.0; vector_data[4] = -1.0; vector_data[5] =  
        1.0;  
    vector_data[6] = 1.0; vector_data[7] = 1.0; vector_data[8] =  
        -1.0;  
    vector_data[9] = 4.0; vector_data[10] = -1.0; vector_data[11]  
        = 4.0;  
}
```

Calling C Function

```
import ctypes
import numpy as np
def load_vectors_from_c():
    lib = ctypes.CDLL('./plane.so')
    # Define the C function signature
    get_vectors_c = lib.get_vectors
    get_vectors_c.argtypes = [ctypes.POINTER(ctypes.c_double)]
    get_vectors_c.restype = None

    vector_data = np.zeros(12, dtype=np.float64)
    get_vectors_c(vector_data.ctypes.data_as(ctypes.POINTER(
        ctypes.c_double)))
    # Reshape the data and return the individual vectors
    vectors_from_c = vector_data.reshape(4, 3)
    a, b, c, v = vectors_from_c[0], vectors_from_c[1],
        vectors_from_c[2], vectors_from_c[3]
    return a, b, c, v
```

Python Code for Plotting

#Code by GVV Sharma

#September 12, 2023

#Revised July 21, 2024

#released under GNU GPL

```
import sys
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
sys.path.insert(0, '/workspaces/urban-potato/matgeo/codes/  
CoordGeo/')
```

```
from call import load_vectors_from_c
```

```
hat_symbol = '\u0302'
```

```
from line.funcs import *
```

```
from triangle.funcs import *
```

```
from conics.funcs import circ_gen
```

```
a, b, c, v = load_vectors_from_c()
```

Python Code for Plotting

```
vx, vy, vz = v[0], v[1], v[2]
j_part_str = f"+ {vy}j{hat_symbol}" if vy >= 0 else f"- {abs(vy)}j{hat_symbol}"
k_part_str = f"+ {vz}k{hat_symbol}" if vz >= 0 else f"- {abs(vz)}k{hat_symbol}"
print(f"The vector in the plane of a and b is: {vx}i{hat_symbol} {j_part_str} {k_part_str}")

# Create a 3D plot
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
origin = [0, 0, 0]

# Plot vectors
vectors = {'a': (a, 'r'), 'b': (b, 'g'), 'c': (c, 'b'), 'v': (v, 'orange')}
for name, (vec, color) in vectors.items():
    x, y, z = int(vec[0]), int(vec[1]), int(vec[2])
    j_part = f"+ {y}j{hat_symbol}" if y >= 0 else f"- {abs(y)}j{hat_symbol}"
```

Python Code for Plotting

```
k_part = f"+ {z}k{hat_symbol}" if z >= 0 else f"- {abs(z)}k{hat_symbol}"
label_text = f'${name}$ = {x}i{hat_symbol} {j_part} {k_part}'
ax.quiver(origin[0], origin[1], origin[2], vec[0], vec[1], vec[2],
          color=color, label=label_text,
          arrow_length_ratio=0.1, linewidth=2)
ax.text(vec[0]*1.1, vec[1]*1.1, vec[2]*1.1, f'${name}$',
        color=color, fontsize=15)
# Plot the plane
x_plane = np.linspace(-5, 5, 10); y_plane = np.linspace(-5, 5, 10)
X, Y = np.meshgrid(x_plane, y_plane)
Z = X
ax.plot_surface(X, Y, Z, alpha=0.2, color='gray')
```

Python Code for Plotting

```
ax.view_init(elev=25, azim=45)
ax.set_xlabel('X'); ax.set_ylabel('Y'); ax.set_zlabel('Z')
ax.set_title('Finding a vector that is in the plane of vectors a
             and b')
ax.legend()
ax.grid(True)
ax.set_xlim([-5, 5]); ax.set_ylim([-5, 5]); ax.set_zlim([-5, 5])
```